Model Assessment: Predicting density and assessing factors driving marbled murrelet distribution in Puget Sound using hierarchical distance sampling

Code by S.J. Gillman

Required Packages

```
library(tidyverse)
library(MCMCvis)
library(nimble)
library(ggplot2)
```

Load Data

Required Files Previously Made or Provided

- `Survey_Grid_Info.RData`
- `CH1_nSQ_results_1.RData`
- `CH1_nSQ_results_2.RData`
- `CH1_nSQ_results_3.RData`

Each of the `CH1_nSQ_results_X.RData` contains the posterior samples, the data and constants that went into the model, and the initial values.

```
load("../DataAnalysis/DataFiles/Survey_Grid_Info.RData")
load("RESULTS/CH1_nSQ_results_1.RData")

load("RESULTS/CH1_nSQ_results_2.RData")

load("RESULTS/CH1_nSQ_results_3.RData")
```

```
chains <- coda::mcmc.list(samples_1[[1]],samples_2[[1]],samples_3[[1]])

result_summary <- MCMCsummary(chains, probs = c(0.025,0.25,0.5,0.75, 0.975)) %>%
  rownames_to_column("parameter") %>%
  mutate(
    variable = sub("\\[.*", "", parameter),
    model_id = gsub("\\]", "", sub(".*\\[", "", parameter)))

round(max(result_summary$Rhat, na.rm =T),2)
```

```
## [1] 1
```

```
pd_dir <- bayestestR::p_direction(chains)


gof_chains <- coda::mcmc.list(samples_1[[2]],samples_2[[2]],samples_3[[2]])
```

```r
results_gof <- MCMCsummary(gof_chains, probs = c(0.025,0.25,0.5,0.75, 0.975))

round(max(results_gof$Rhat, na.rm =T),2)
```

```
## [1] 1.17
```

```r
post_gof <- as.matrix(rbind(samples_1[[2]], samples_2[[2]], samples_3[[2]]))
```

N GoF

```r
N_sub <- post_gof %>%
  as.data.frame() %>%
  dplyr::select(contains("N[")) %>%
  dplyr::select(-c(contains("rN"), contains("total_expected"))) %>%
  as.matrix()

rN_sub <- post_gof %>%
  as.data.frame() %>%
  dplyr::select(contains("rN")) %>%
  as.matrix()

lambda_sub <- post_gof %>%
  as.data.frame() %>%
  dplyr::select(contains("lambda")) %>%
  as.matrix()


# result matrix
N_new <- matrix(0, nrow = nimConstants$nS, ncol = nrow(N_sub))

# Loop over sites
for (r in 1:nimConstants$nS) {
  my_idx <- nimConstants$MONTHYEAR[r]

  # Extract vectors for all iterations at once
  rN_vec <- rN_sub[, my_idx]
  lambda_vec <- lambda_sub[, r]

  # Calculate p
  prob_p <- rN_vec / (rN_vec + lambda_vec)

  # Create new N estimates
  N_new[r, ] <- rnbinom(nrow(N_sub), size = rN_vec, prob = prob_p)
}

# Residuals for 'observed'
FT_obs <- colSums((sqrt(t(N_sub)) - sqrt(t(lambda_sub)))^2)
# Residuals for 'new'
FT_sim <- colSums((sqrt(N_new) - sqrt(t(lambda_sub)))^2)
```

```r
# Posterior Predictive P-value
round(mean(FT_sim > FT_obs),2)
```

```
## [1] 0.89
```

GS GoF

```r
gs_sub <- post_gof %>%
  as.data.frame() %>%
  dplyr::select(contains("gs.lam")) %>%
  as.matrix()

# result matrix
GS_new <- matrix(0, nrow = nimConstants$nS, ncol = nrow(gs_sub))
E_gs <- matrix(0, nrow = nimConstants$nS, ncol = nrow(gs_sub))

for (r in 1:nimConstants$nS) {
  my_idx <- nimConstants$MONTHYEAR[r]

  gs_vec <- gs_sub[, my_idx]
  E_gs[r, ] <- nimData$C[r] * gs_sub[, my_idx]
  GS_new[r, ] <- rpois(nrow(gs_sub), E_gs[r, ])
}

# Residuals for 'observed'
FT_obs <- colSums((sqrt(nimData$GS) - sqrt(E_gs))^2)
# Residuals for 'new'
FT_sim <- colSums((sqrt(GS_new) - sqrt(E_gs))^2)

# Posterior Predictive P-value
mean(FT_sim > FT_obs)
```

```
## [1] 1
```

Counts GoF

```r
pcap_sub <- post_gof %>%
  as.data.frame() %>%
  dplyr::select(contains("pcap")) %>%
  as.matrix()

C_actual <- nimData$C


C_new <- matrix(0, nrow = nimConstants$nS, ncol = nrow(N_sub))
C_expected <- matrix(0, nrow = nimConstants$nS, ncol = nrow(N_sub))

for (r in 1:nimConstants$nS) {
  N_vec    <- N_sub[, r]
```

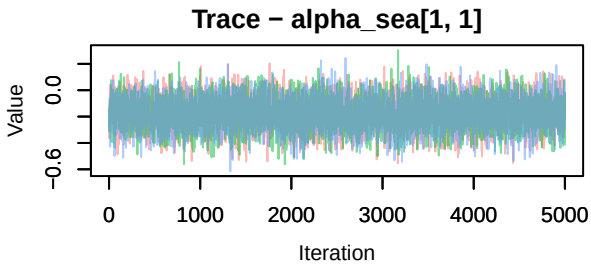
```
  pcap_vec <- pcap_sub[, r]

  # expected Counts
  C_expected[r,] <- N_sub[, r] * pcap_sub[, r]
  # Create new C estimates
  C_new[r, ] <- rbinom(nrow(N_sub), size = N_vec, prob = pcap_vec)
}

# Residuals for 'observed'
FT_obs <- colSums((sqrt(nimData$C) - sqrt(C_expected))^2)
# Residuals for 'new'
FT_sim <- colSums((sqrt(C_new) - sqrt(C_expected))^2)

#Posterior Predictive P-value
round(mean(FT_sim > FT_obs), 2)
```

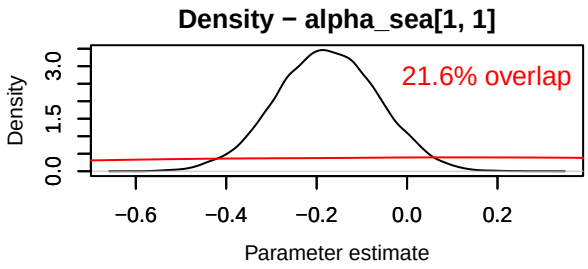
```
## [1] 0.52
```

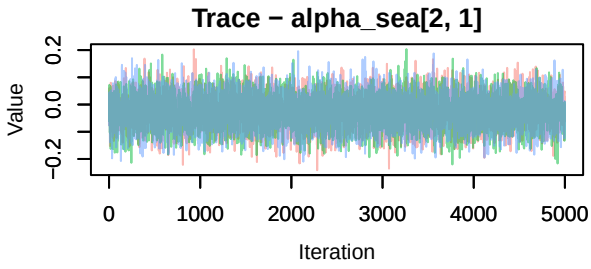
Prior posterior overlap

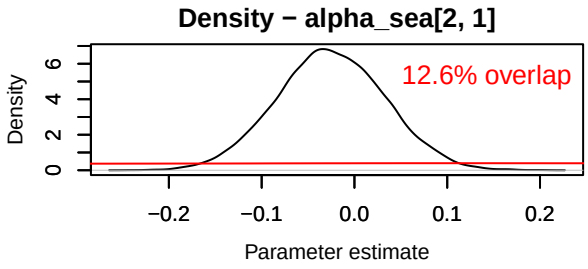
```
PR <- rnorm(15000, 0, 1)

MCMCtrace(chains, params = 'alpha_sea', priors = PR, pdf = FALSE)
```

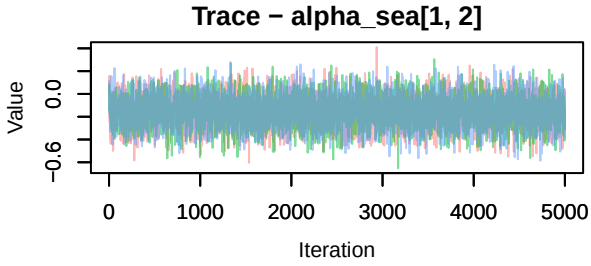
```
## Warning in MCMCtrace(chains, params = "alpha_sea", priors = PR, pdf = FALSE):
## Only one prior specified for > 1 parameter. Using a single prior for all
## parameters.
```

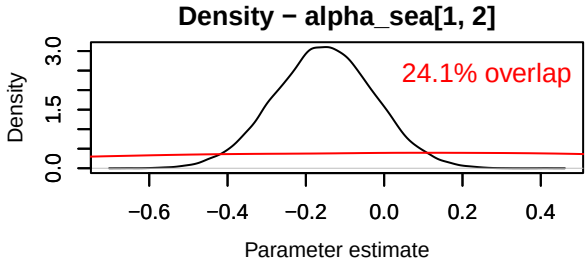

Trace − alpha_sea[1, 1]
Value
Iteration

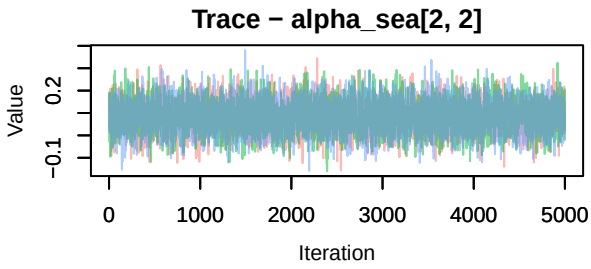

Density − alpha_sea[1, 1]
21.6% overlap
Density
Parameter estimate

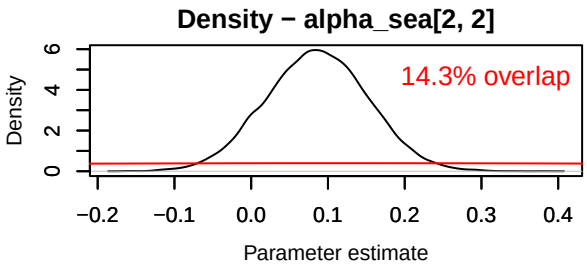

Trace − alpha_sea[2, 1]
Value
Iteration

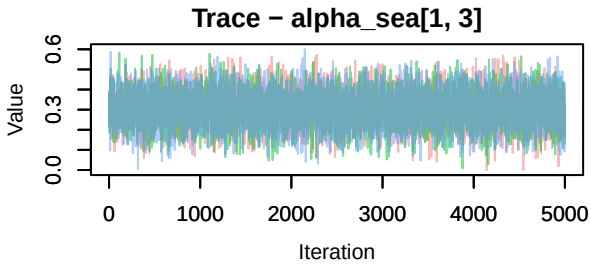

Density − alpha_sea[2, 1]
12.6% overlap
Density
Parameter estimate

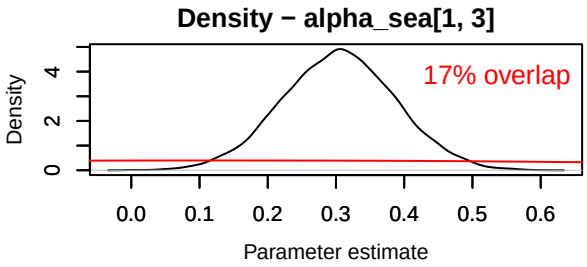

Trace − alpha_sea[1, 2]
Value
Iteration

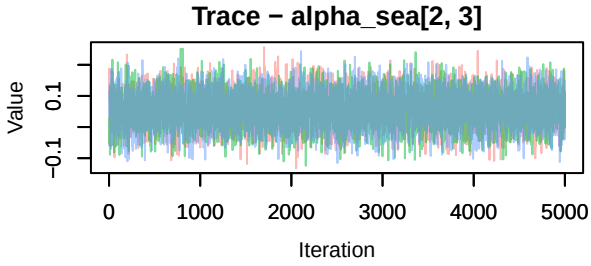

Density − alpha_sea[1, 2]
24.1% overlap
Density
Parameter estimate

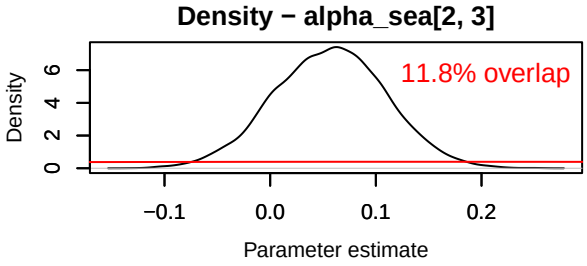

Trace − alpha_sea[2, 2]
Value
−0.1
0.2
0
1000
2000
3000
4000
5000
Iteration

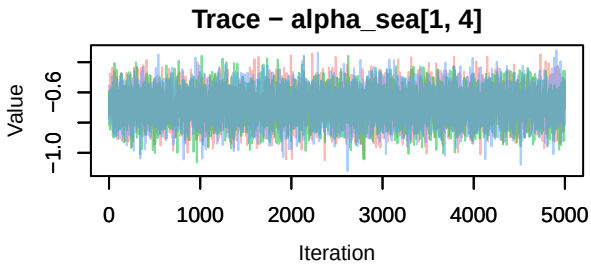

Density − alpha_sea[2, 2]
Density
0
2
4
6
14.3% overlap
−0.2
−0.1
0.0
0.1
0.2
0.3
0.4
Parameter estimate

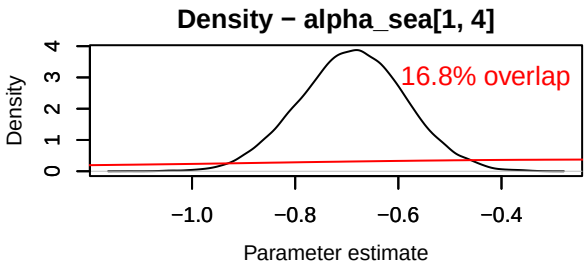

Trace − alpha_sea[1, 3]
Value
0.0
0.3
0.6
0
1000
2000
3000
4000
5000
Iteration

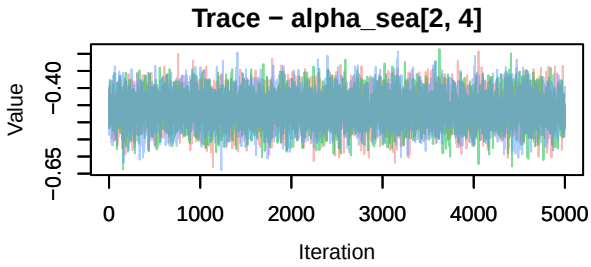

Density − alpha_sea[1, 3]
Density
0
2
4
17% overlap
0.0
0.1
0.2
0.3
0.4
0.5
0.6
Parameter estimate

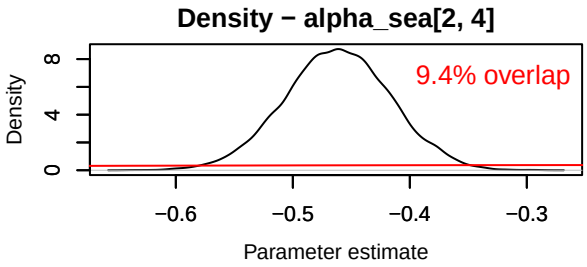

Trace − alpha_sea[2, 3]
Value
−0.1
0.1
0
1000
2000
3000
4000
5000
Iteration

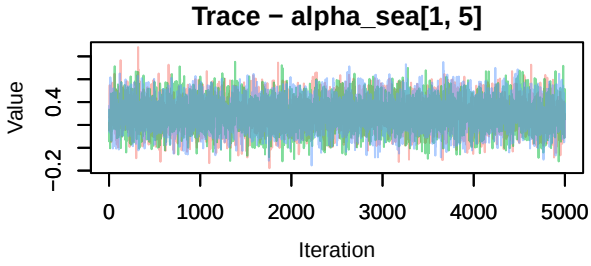

Density − alpha_sea[2, 3]
Density
0
2
4
6
11.8% overlap
−0.1
0.0
0.1
0.2
Parameter estimate

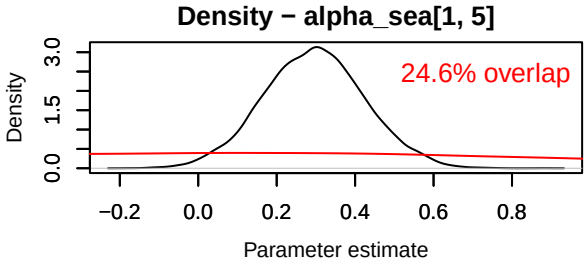

Trace − alpha_sea[1, 4]
Value
−1.0
−0.6
0
1000
2000
3000
4000
5000
Iteration

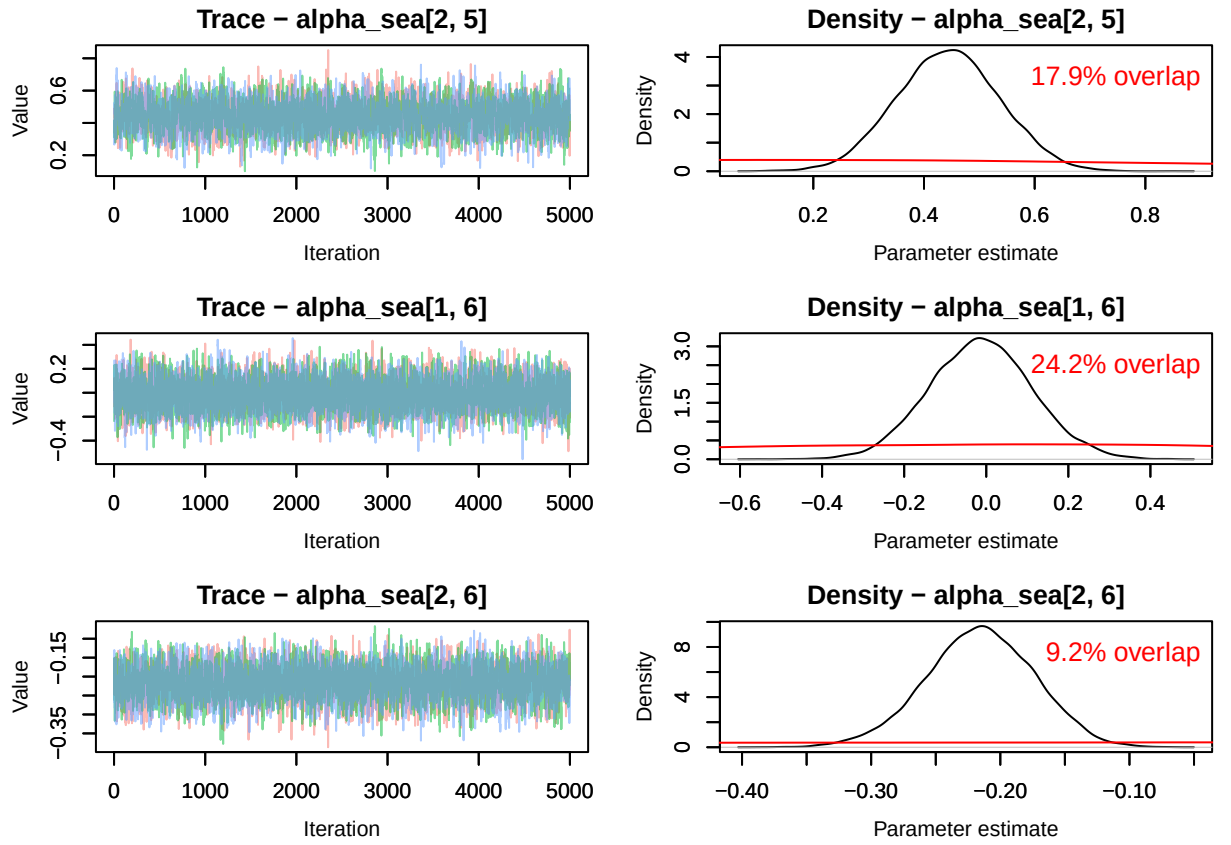

Density − alpha_sea[1, 4]
Density
0
1
2
3
4
16.8% overlap
−1.0
−0.8
−0.6
−0.4
Parameter estimate


Trace − alpha_sea[2, 4]
Value
−0.65
−0.40
0
1000
2000
3000
4000
5000
Iteration


Density − alpha_sea[2, 4]
Density
0
4
8
9.4% overlap
−0.6
−0.5
−0.4
−0.3
Parameter estimate

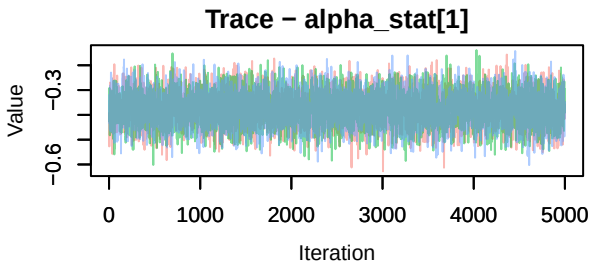

Trace − alpha_sea[1, 5]
Value
−0.2
0.4
0
1000
2000
3000
4000
5000
Iteration

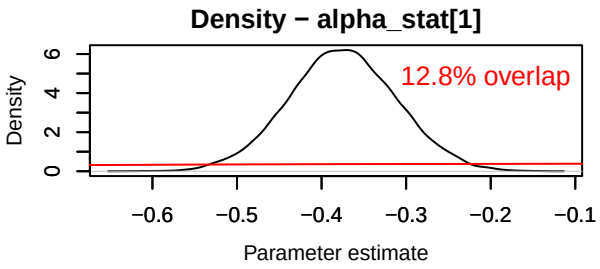

Density − alpha_sea[1, 5]
Density
0.0
1.5
3.0
24.6% overlap
−0.2
0.0
0.2
0.4
0.6
0.8
Parameter estimate

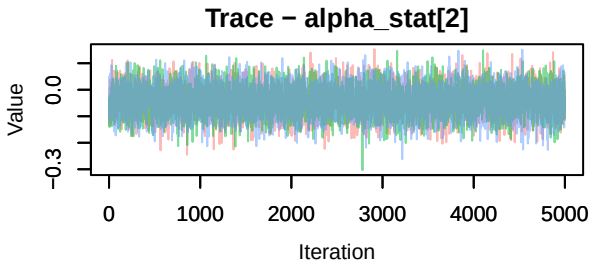


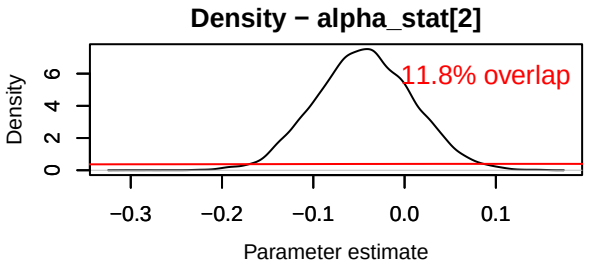

```
MCMCtrace(chains, params = 'alpha_stat', priors = PR, pdf = FALSE)
```

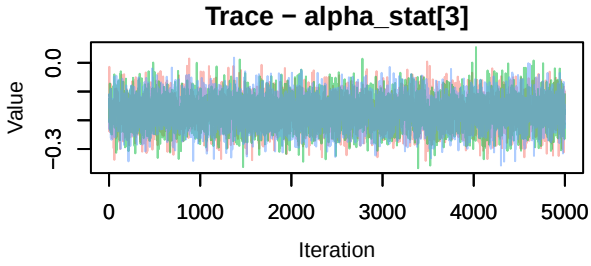
```
## Warning in MCMCtrace(chains, params = "alpha_stat", priors = PR, pdf = FALSE):
## Only one prior specified for > 1 parameter. Using a single prior for all
## parameters.
```

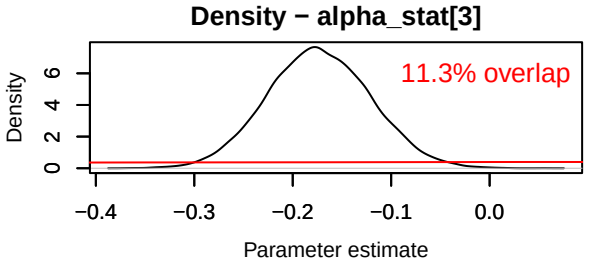

Trace − alpha_stat[1]
Value
−0.3
−0.6
0
1000
2000
3000
4000
5000
Iteration

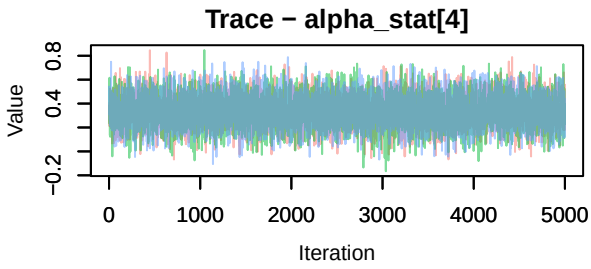

Density − alpha_stat[1]
Density
0
2
4
6
12.8% overlap
−0.6
−0.5
−0.4
−0.3
−0.2
−0.1
Parameter estimate

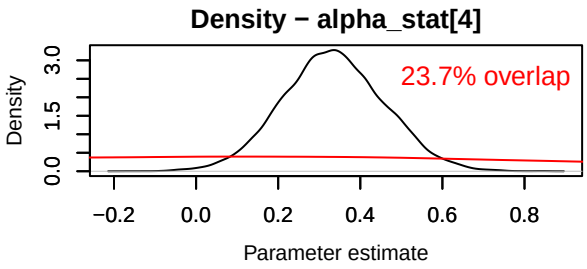

Trace − alpha_stat[2]
Value
0.0
−0.3
0
1000
2000
3000
4000
5000
Iteration

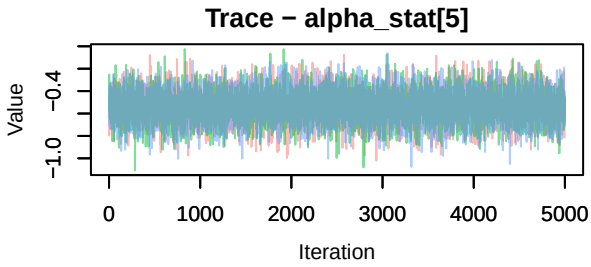

Density − alpha_stat[2]
Density
0
2
4
6
11.8% overlap
−0.3
−0.2
−0.1
0.0
0.1
Parameter estimate

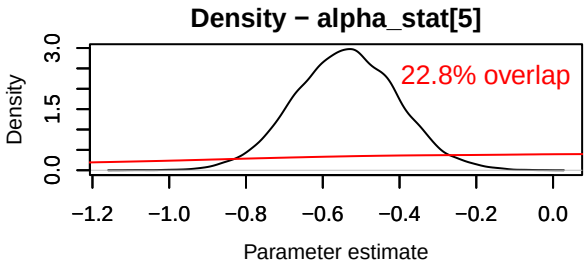

Trace − alpha_stat[3]
Value
0.0
−0.3
0
1000
2000
3000
4000
5000
Iteration

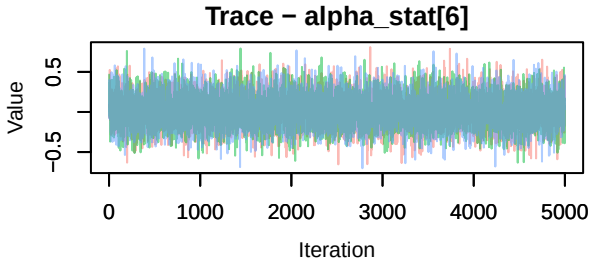

Density − alpha_stat[3]
Density
0
2
4
6
11.3% overlap
−0.4
−0.3
−0.2
−0.1
0.0
Parameter estimate

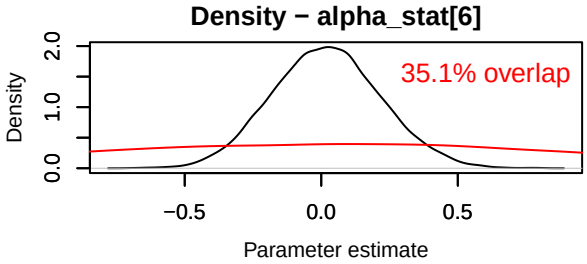

Trace − alpha_stat[4]
Value
Iteration

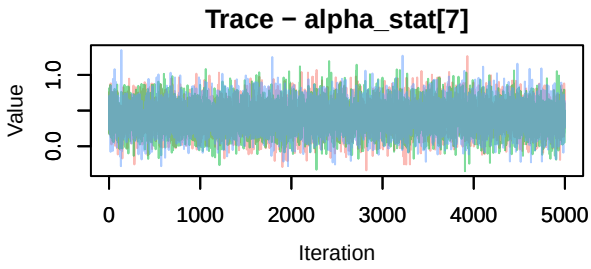

Density − alpha_stat[4]
23.7% overlap
Density
Parameter estimate

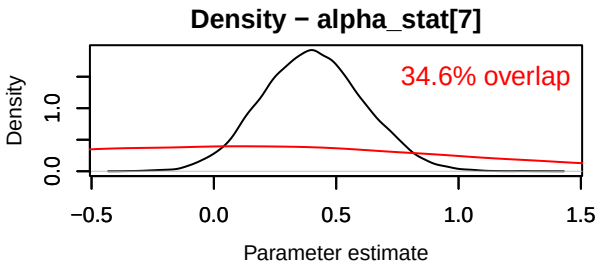

Trace − alpha_stat[5]
Value
Iteration

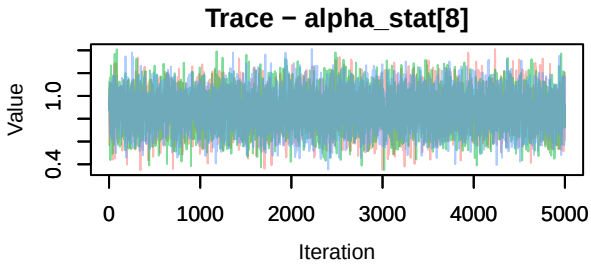

Density − alpha_stat[5]
22.8% overlap
Density
Parameter estimate

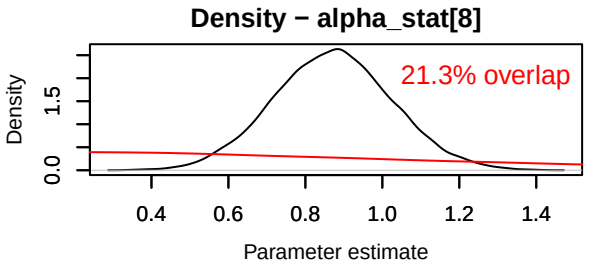

Trace − alpha_stat[6]
Value
Iteration

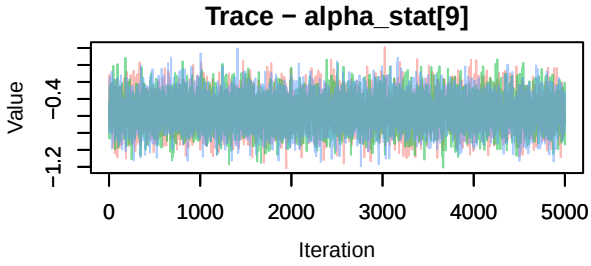

Density − alpha_stat[6]
35.1% overlap
Density
Parameter estimate

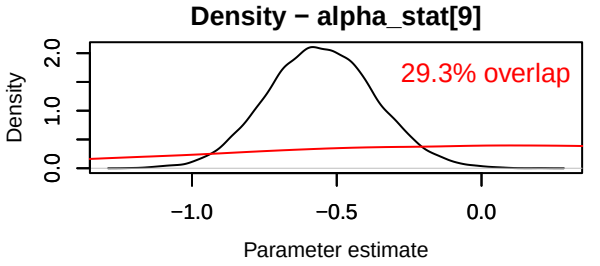

Trace − alpha_stat[7]
Value
1.0
0.0
0
1000
2000
3000
4000
5000
Iteration

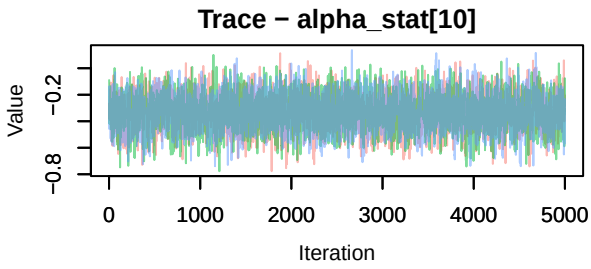

Density − alpha_stat[7]
34.6% overlap
Density
1.0
0.0
−0.5
0.0
0.5
1.0
1.5
Parameter estimate

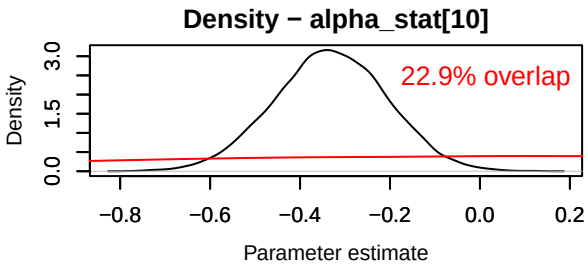

Trace − alpha_stat[8]
Value
1.0
0.4
0
1000
2000
3000
4000
5000
Iteration

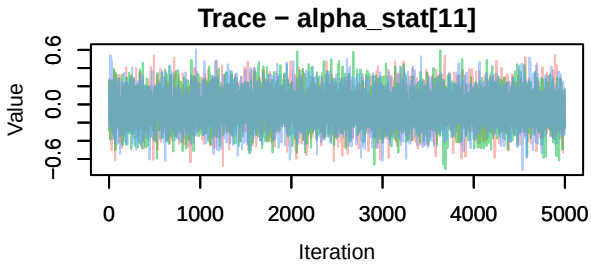

Density − alpha_stat[8]
21.3% overlap
Density
1.5
0.0
0.4
0.6
0.8
1.0
1.2
1.4
Parameter estimate

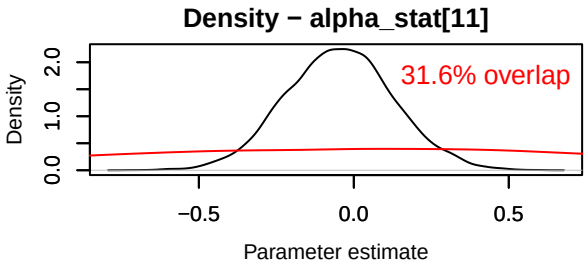

Trace − alpha_stat[9]
Value
−0.4
−1.2
0
1000
2000
3000
4000
5000
Iteration

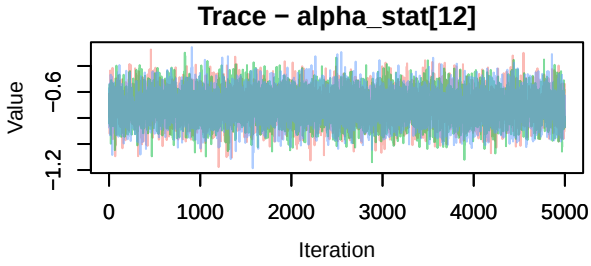

Density − alpha_stat[9]
29.3% overlap
Density
2.0
1.0
0.0
−1.0
−0.5
0.0
Parameter estimate

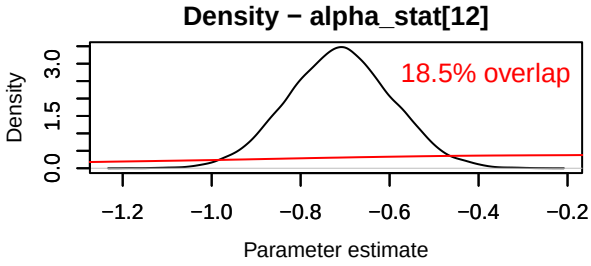

Trace − alpha_stat[10]
Value
−0.8
−0.2
0
1000
2000
3000
4000
5000
Iteration

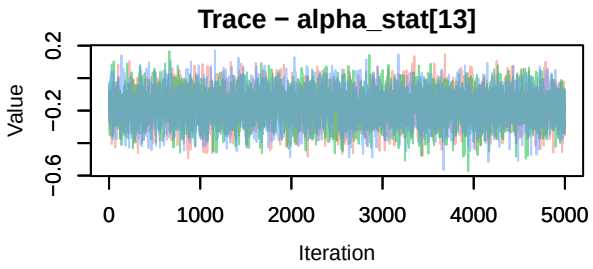

Density − alpha_stat[10]
Density
0.0
1.5
3.0
22.9% overlap
−0.8
−0.6
−0.4
−0.2
0.0
0.2
Parameter estimate

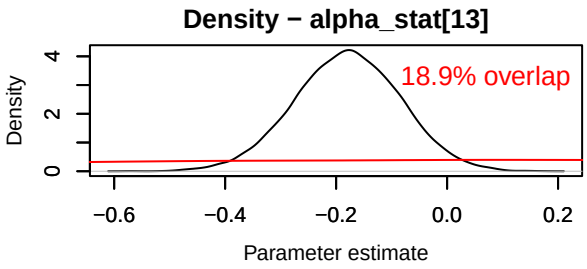

Trace − alpha_stat[11]
Value
−0.6
0.0
0.6
0
1000
2000
3000
4000
5000
Iteration

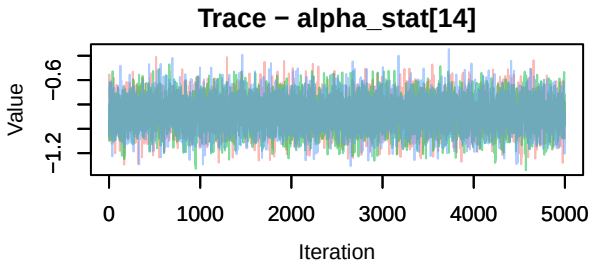

Density − alpha_stat[11]
Density
0.0
1.0
2.0
31.6% overlap
−0.5
0.0
0.5
Parameter estimate

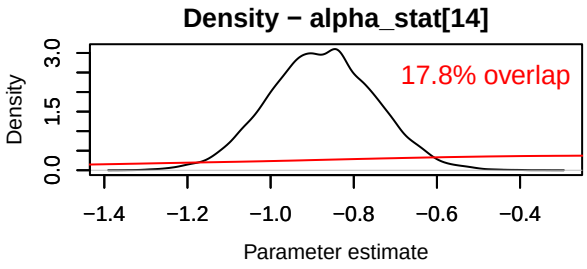

Trace − alpha_stat[12]
Value
−1.2
−0.6
0
1000
2000
3000
4000
5000
Iteration

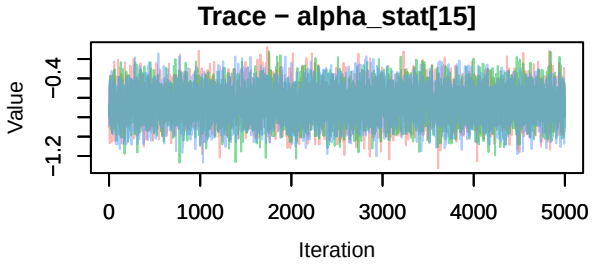

Density − alpha_stat[12]
Density
0.0
1.5
3.0
18.5% overlap
−1.2
−1.0
−0.8
−0.6
−0.4
−0.2
Parameter estimate

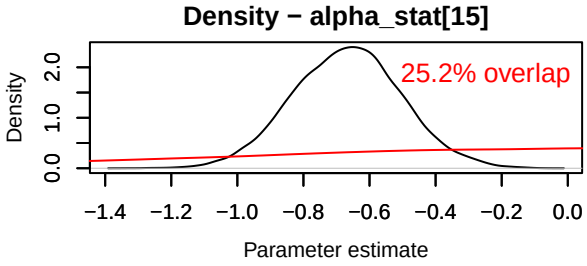

Trace − alpha_stat[13]
Value
−0.6
−0.2
0.2
0
1000
2000
3000
4000
5000
Iteration

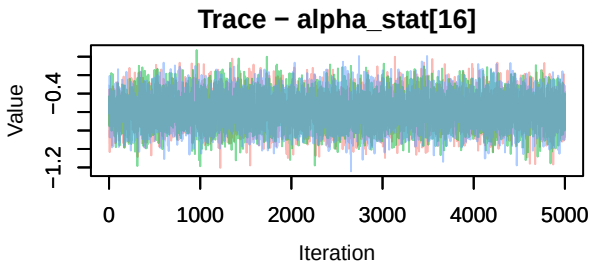

Density − alpha_stat[13]
Density
0
2
4
18.9% overlap
−0.6
−0.4
−0.2
0.0
0.2
Parameter estimate

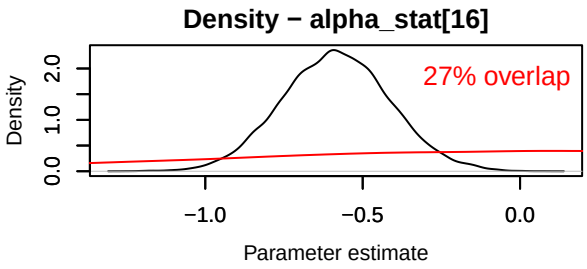

Trace − alpha_stat[14]
Value
−1.2
−0.6
0
1000
2000
3000
4000
5000
Iteration

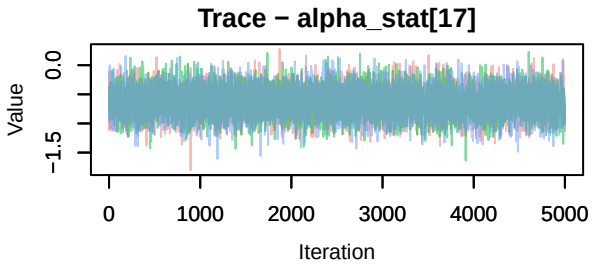

Density − alpha_stat[14]
Density
0.0
1.5
3.0
17.8% overlap
−1.4
−1.2
−1.0
−0.8
−0.6
−0.4
Parameter estimate

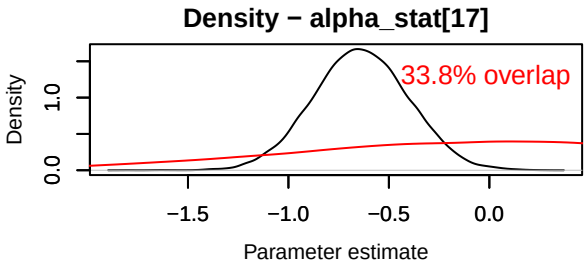

Trace − alpha_stat[15]
Value
−1.2
−0.4
0
1000
2000
3000
4000
5000
Iteration

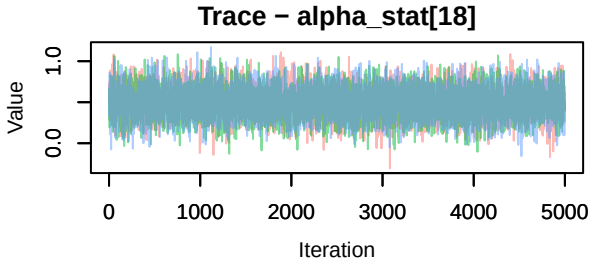

Density − alpha_stat[15]
Density
0.0
1.0
2.0
25.2% overlap
−1.4
−1.2
−1.0
−0.8
−0.6
−0.4
−0.2
0.0
Parameter estimate

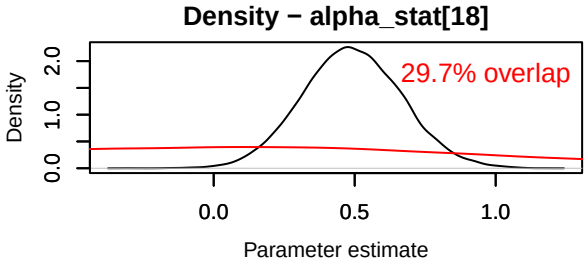

Trace − alpha_stat[16]
Value
−1.2
−0.4
0
1000
2000
3000
4000
5000
Iteration


Density − alpha_stat[16]
Density
0.0
1.0
2.0
27% overlap
−1.0
−0.5
0.0
Parameter estimate


Trace − alpha_stat[17]
Value
−1.5
0.0
0
1000
2000
3000
4000
5000
Iteration


Density − alpha_stat[17]
Density
0.0
1.0
33.8% overlap
−1.5
−1.0
−0.5
0.0
Parameter estimate


Trace − alpha_stat[18]
Value
0.0
1.0
0
1000
2000
3000
4000
5000
Iteration


Density − alpha_stat[18]
Density
0.0
1.0
2.0
29.7% overlap
0.0
0.5
1.0
Parameter estimate